Trabajo Practico Algoritmos y Estructura de Datos

Integrantes:

1. Apellido y nombre: Quentasi Cejas Matias Gustavo

   Usuario de GitHub: mquentasicejas

   Legajo: 235.529-2

   Correo: mquentasicejas@frba.utn.edu.ar

   Tareas: Operación 1-2-3-4-5-6

2. Apellido y nombre: Fernandez Maran Lorenzo

   Usuario de GitHub: lfernandezmaran

   Legajo: 235.484-6

   Correo: lfernandezmaran@frba.utn.edu.ar

   Tareas: Operación 7

Docente: Pablo Damian Mendez

Comisión: K1103.

Fecha de entrega: 2/11

Ciclo lectivo: 2026

Primero definimos ambos struct que nos da la consigna que son el struct Orden y OrdenArchivo. El struct Orden guarda lo que tiene que hacer el dron en un casillero mientras que en el struct OrdenArchivo tengo que poner las coordenadas de x e y manualmente.

Luego para simplificar código hacemos las siguientes funciones: **encontrarRegistro**, **pedirCoordenadas**, **pedirAtaques** y **pedirCoordenadasSiguiente** para simplificar código y no codear a cada rato cin y cout. **mostrarDatos** para mostrar los datos de un registro y **mostrarDatosSinXeY** para mostrar los datos de una tabla.

OPERACION 1 (en esta operación tuve bastante ayuda de Claude): Primero lee un archivo de ataque y lo pasa a la tabla en memoria, pero solo si la ruta del dron es valida. Primero pide el nombre del archivo y lo abre. Si no existe, avisa y termina.

Luego lee los registros uno por uno, por cada registro revisa que las coordenadas esten dentro de la tabla, que no se repitan, que el despegue no traiga ataques y que no tengan el mismo valor el ataqueKamikaze y el aterrizaje. Si se cargan los archivos correctamente entonces lo guarda en la posición de la tabla que tiene sus coordenadas x e y, y va

contando cuantos despegues y cuantos finales (aterrizaje o ataqueKamikaze) encontró. Cuando termina de leer verifica que haya un despegue y un final.

Por ultimo recorre la ruta: arranca en el despegue y va de casillero en casillero siguiente las coordenadas “siguiente”. Verifica que cada paso sea a un casillero cercano o vecino, que no haya huecos y que todos los registros del archivo formen parte de esa ruta.

Si algo falla en cualquier momento, avisa cual fue el error, vacia la tabla y dice que no se cargó el archivo. Si todo esta bien, informa cuantos archivos se cargaron.

OPERACIÓN 2: Esta operación recorre la tabla con dos for, y, por cada casillero que tiene datos, muestra sus valores por pantalla. Si un casillero de la tabla es el final entonces escribe “Fin del archivo”, sino, muestra a que coordenadas va despues. Cada 20 casilleros mostrados avisa y espera una tecla con getch() antes de seguir. Si no había ningún dato avísa por pantalla: “No hay datos en la tabla”.

OPERACIÓN 3: Básicamente este procedimiento nos sirve crear un archivo nuevo con modo de apertura en “wb”, se dan valores a los ataques correspondientes del struct OrdenArchivo. Tiene las mismas restricciones que las operaciones anteriores.

OPERACIÓN 4: La operación 4 lo que hace es pedirte cuales son las coordenadas del registro que queres modificar, ahí entra el procedimiento buscarRegistro. Si el registro es encontrado entonces tenes que ponerle los nuevos valores al registro, en caso de que no encuentre el archivo te sale por pantalla “El archivo no se encontró” y retorna.

OPERACIÓN 5: En esta operación lo que hacemos primero es pedir las coordenadas con el procedimiento pedirCoordenadas, luego en caso de que haya encontrado datos en esa posición de la tabla lo muestra por pantalla, en caso de que no, te muestra por pantalla “Estas coordenadas no tienen datos en memoria cargada”. Luego lo mismo, vuelvo a pedirle al usuario que ingrese los nuevos valores.

OPERACIÓN 6: Lo que hace esta operación es toma lo que hay en la tabla en memoria y lo guarda en un archivo nuevo. Primero pide el nombre del archivo y lo abre en “wb”. Si no puede, avisa y termina.

Recorre la tabla con dos for. Por cada casillero con datos, arma un OrdenArchivo: pone buffer.x = x y buffer.y = y. Lo escribo con fwrite y cuenta cuantos archivos se guardaron.

En esta operación solo se guardan casilleros con datos. El orden de los registros se guardan recorriendo x y despues y, no en el orden de la ruta del dron. Si la tabla esta vacia, se crea el archivo igual pero vacío y al final se avisa.

OPERACIÓN 7: Al no haber tenido conocimientos previos sobre html, esta operación se nos complicó en sus inicios. El ejemplo que el profesor nos dejó en el pdf de la consigna fue clave como punto de referencia para entender cómo íbamos a crear la tabla. A la hora de buscar el archivo el cual quiere ser transformado en html, se debe escribir .html al final del nombre, quedando “Nombre_del_Archivo.html”, de esta forma se evita que se reescriba el archivo original, y permite acceder de manera rápida a la tabla.

El diccionario de la tabla sería

D: despegue. Es el inicio del recorrido del dron.

1: soltar granada 1. El primero de los tres ataques que puede realizar el dron.

2: soltar granada 2. El segundo de los tres ataques que puede realizar el dron.

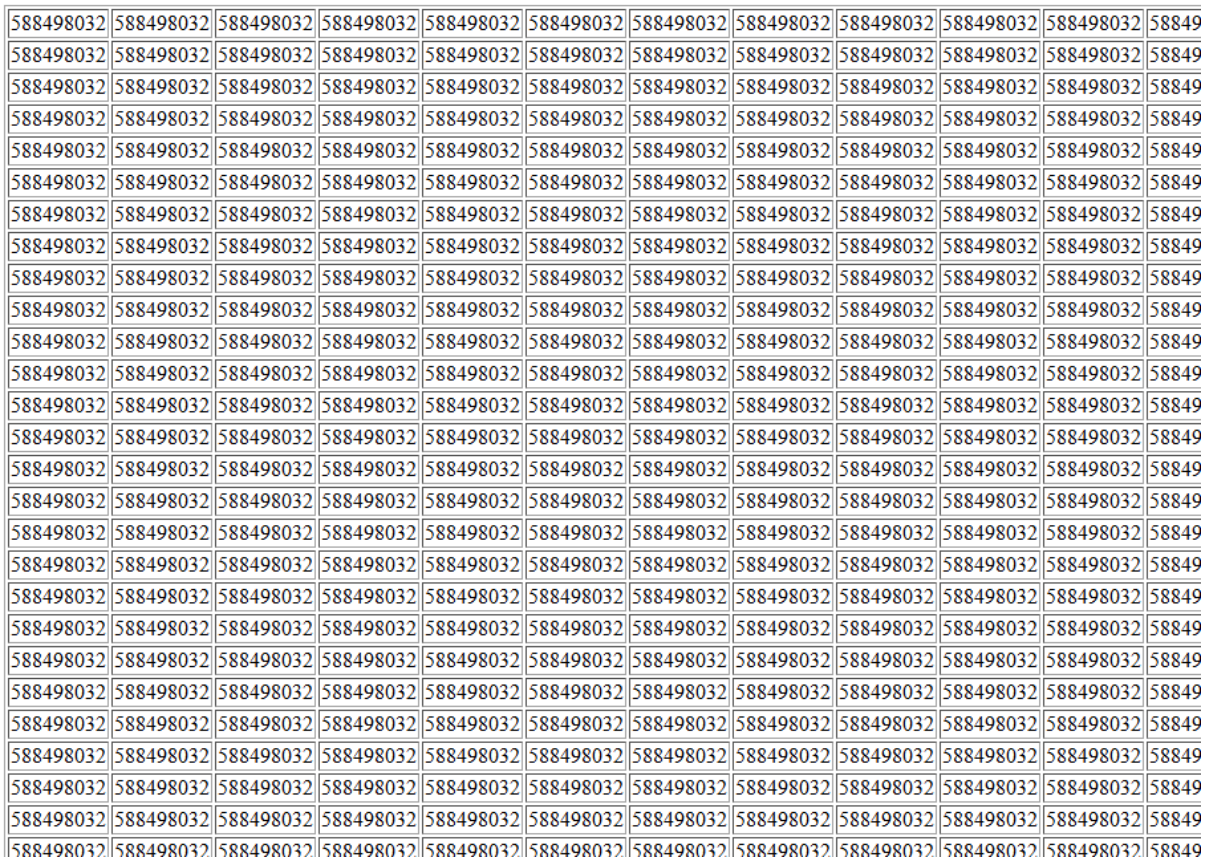
E: espera. Indica la espera, el tiempo de espera solo aparecerá a la hora de ejecutar la operación 2, buscando la casilla donde fue indicada la espera.

K: ataque kamikaze. El último de los tres ataques que puede realizar el dron, además de ser una de las dos alternativas para finalizar el recorrido.

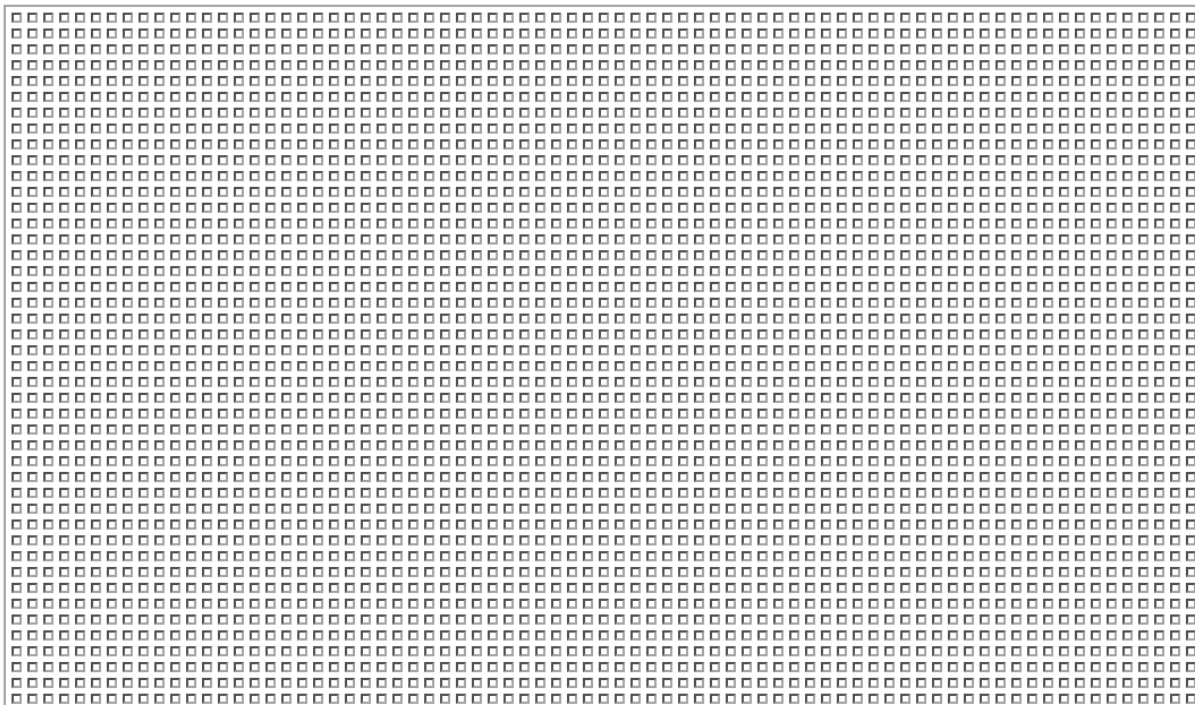
A: aterrizaje. La última de las dos alternativas para finalizar el recorrido.

M: movimiento. En caso de no ejecutar ningún ataque, espera o aterrizaje, la casilla será marcada como movimiento, sin tener ninguna acción en esa misma.

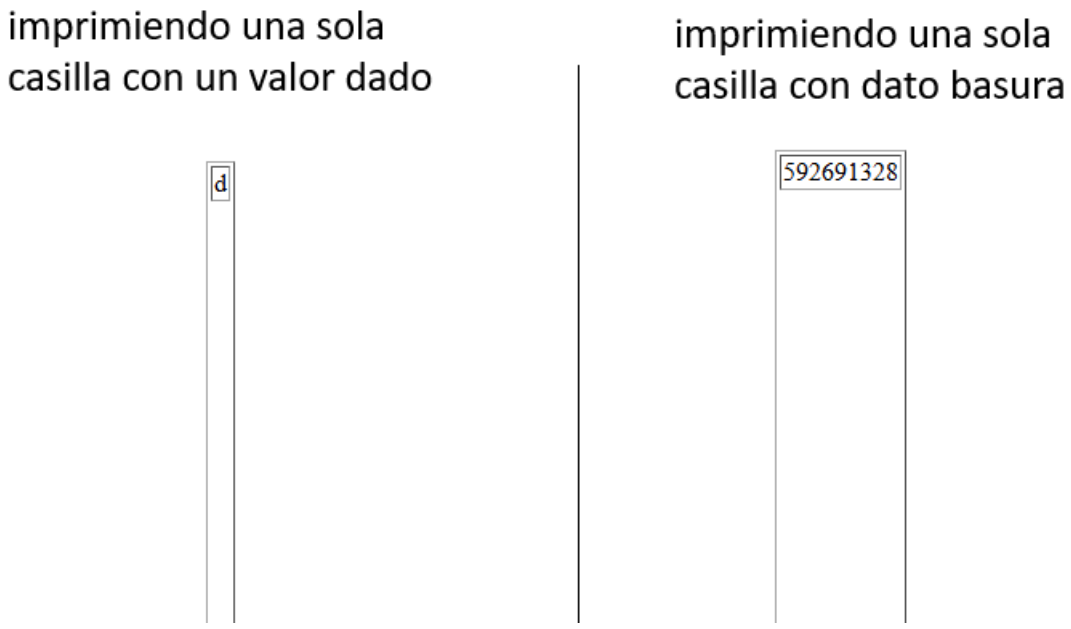
- La primera vez que creamos la tabla, nos encontramos con el problema de que solo se mostraban datos basura, como vemos en la siguiente imagen.



- Buscando eliminar los datos basura, pudimos conseguir una tabla vacia, que no nos servía ya que seguía sin mostrar los datos deseados.



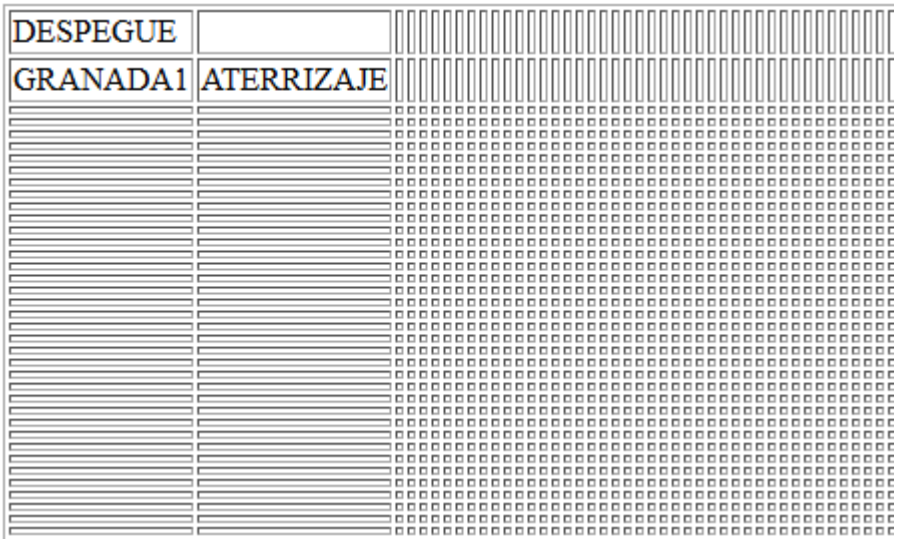
- A la hora de buscar que solo cree los datos deseados, nos encontramos con problemas para definir la tabla.



- Decidimos por definir que si en la casilla no se declaraba nada, se imprimiría la palabra “NADA”. Para esto, utilizamos unos condicionales que solo permitían imprimir el valor verdadero para las operaciones, el “1”. A pesar de esto, todavía nos faltaba especificar qué significaba cada 1.

1	1	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA
NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA	NADA

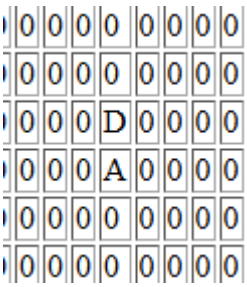
- Decidimos nombrar las operaciones por sus respectivos nombres y a los valores vacíos como un espacio en blanco, pero nos generaba problemas por el tamaño de las palabras comparadas con los diminutos bloques vacíos.



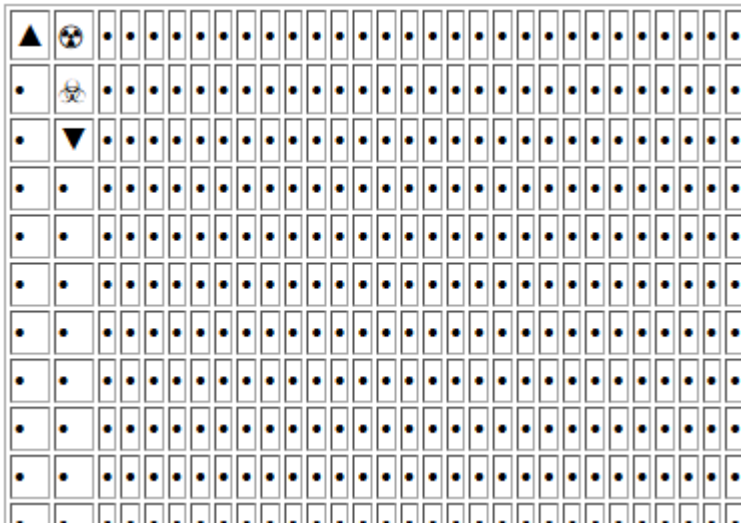
- Terminamos optando por definir una serie de condicionales antes de que se impriman los valores en las casillas, esto con la finalidad de que solo imprimiría lo que haya sido declarado en esa casilla. Por ejemplo, si se despegaba en la casilla (2,2), se escribiría una D en la casilla (2,2).

```
if (Buffer.despegue == 1)
{
    fprintf(archivo, "<td>   D   </td>", tabla[fila][columna]);
}
else
{
    if (Buffer.aterrizaje == 1)
    {
        fprintf(archivo, "<td>   A   </td>", tabla[fila][columna]);
    }
    else
    {
        if (Buffer.soltarGranada1 == 1)
        {
            fprintf(archivo, "<td>   1   </td>", tabla[fila][columna]);
        }
        else
        {
            if (Buffer.soltarGranada2 == 1)
            {
                fprintf(archivo, "<td>   2   </td>", tabla[fila][columna]);
            }
            else
            {
                if (Buffer.ataqueKamikaze == 1)
                {
                    fprintf(archivo, "<td>   K   </td>", tabla[fila][columna]);
                }
                else
                {
                    if (Buffer.espera != 0)
                    {
                        fprintf(archivo, "<td> E </td>", tabla[fila][columna]);
                    }
                    else
                    {
                        if (Buffer.siguientex != 0 && Buffer.siguientey != 0)
                        {
                            fprintf(archivo, "<td> M </td>", tabla[fila][columna]);
                        }
                        else
                        {
                            fprintf(archivo, "<td>   0   </td>", tabla[fila][columna]);
                        }
                    }
                }
            }
        }
    }
}
```

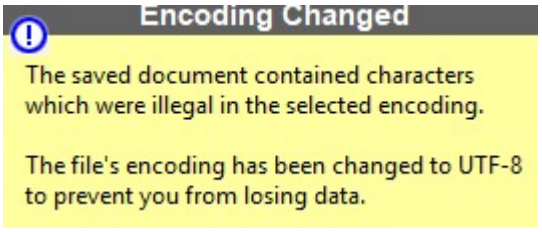
- A la hora de ponerlo en prueba, fue un éxito rotundo.



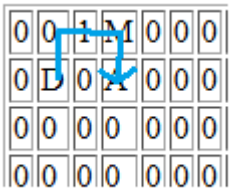
- Quisimos mejorar el código, dándole un toque más decorado. Al no saber cómo dibujar las operaciones, para que quede estético como en el ejemplo del pdf, decidimos que cada operación tendría un símbolo de la página piliapp.com. Nos terminaría quedando de esta forma.

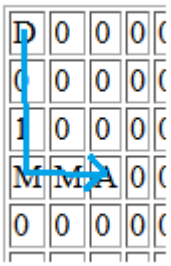


- Pero a la hora de ejecutar el programa luego de haber hecho algún cambio, se nos presentaba este error en la esquina inferior derecha. Básicamente, como el código estaba configurado de una forma que no podía leer los símbolos que nosotros ingresábamos. Para solucionar este error, se había cambiado a la forma de codificación UTF-8, la cual se permitía leer los símbolos. Como nosotros no estábamos seguros de que esto fuera algo válido para el trabajo práctico, decidimos volver a escribirlo en letras y números, evitando algún inconveniente.



- Un problema se nos había surgido, si la operación de movimiento estaba en una fila/columna 0, el primer valor lo imprimía como 0, en lugar de la M de movimiento.

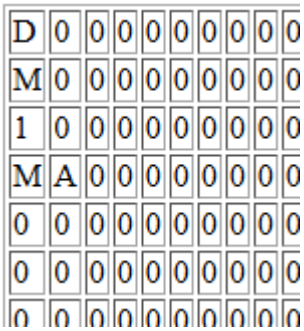




- Al final, esto se debía a que el condicional de movimiento había sido escrito con un &&, en lugar de un ||. Probamos con cambiarlo.

```
if (Buffer.despegue == 1)
{
    fprintf(archivo, "<td>   D   </td>", tabla[fila][columna]);
}
else
{
    if (Buffer.aterrizaje == 1)
    {
        fprintf(archivo, "<td>   A   </td>", tabla[fila][columna]);
    }
    else
    {
        if (Buffer.soltarGranada1 == 1)
        {
            fprintf(archivo, "<td>   1   </td>", tabla[fila][columna]);
        }
        else
        {
            if (Buffer.soltarGranada2 == 1)
            {
                fprintf(archivo, "<td>   2   </td>", tabla[fila][columna]);
            }
            else
            {
                if (Buffer.ataqueKamikaze == 1)
                {
                    fprintf(archivo, "<td>   K   </td>", tabla[fila][columna]);
                }
                else
                {
                    if (Buffer.espera != 0)
                    {
                        fprintf(archivo, "<td> E </td>", tabla[fila][columna]);
                    }
                    else
                    {
                        if (Buffer.siguientex != 0 || Buffer.siguientey != 0)
                        {
                            fprintf(archivo, "<td> M </td>", tabla[fila][columna]);
                        }
                        else
                        {
                            fprintf(archivo, "<td>   0   </td>", tabla[fila][columna]);
                        }
                    }
                }
            }
        }
    }
}
```

- Finalmente, el problema había dejado de presentarse, llegando hasta nuestro objetivo buscado.



*** Detalle de la operación 7:** Es recomendable tener a mano el registro con las órdenes ingresadas. De no ser así, no hay forma de orientarse sobre hacia dónde va el dron, ya que la tabla no presenta un indicador que muestre el orden de inicio a final. Es un error que puede ser notable si la persona no sabe las casillas por las que el dron pasará, habiendo situaciones en las que se puedan confundir con el recorrido del dron. Intentamos solucionarlo buscando distintas alternativas pero no logramos resolverlo. Lo único que se tiene que dar por sabido, es que se iniciará el recorrido en la D de despegue y se terminará en la A de aterrizaje o la K de kamikaze.

